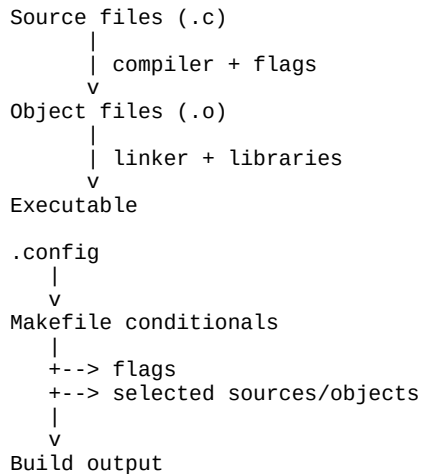


Makefile Concepts — Embedded Linux Practical Summary

A practical reference for reading and making small changes to existing Makefiles, focused on Embedded Linux board bring-up and configuration.

1. Overall Build Flow



2. Basic Makefile Structure

```
TARGET = SampleKlcTest
CC = gcc
OBJ = main.o add.o sub.o

all: $(TARGET)

$(TARGET): $(OBJ)
<TAB>$(CC) $^ -o $@

%.o: %.c
<TAB>$(CC) $(CFLAGS) -c $< -o $@

clean:
<TAB>rm -f *.o $(TARGET)
```

Target = what Make builds. **Prerequisites** = dependencies. **Recipe** = commands used to build the target. Recipe lines normally start with a TAB.

3. Variables and Automatic Variables

```
TARGET = SampleKlcTest
CC = gcc
SRC = main.c add.c sub.c
OBJ = main.o add.o sub.o

$(TARGET) -> SampleKlcTest
$(CC)      -> gcc

$@ -> current target
$< -> first prerequisite
$^ -> all prerequisites
```

For `$(TARGET): $(OBJ)`, `$@` is `SampleKlcTest` and `$^` is the complete object list. For `main.o: main.c`, `$<` is `main.c`.

4. Pattern Rule

```
%.o: %.c
<TAB>$(CC) $(CFLAGS) -c $< -o $@
```

The `%` is a pattern placeholder. The same rule can handle `main.c → main.o`, `add.c → add.o`, `sub.c → sub.o`, etc.

5. Common Make Functions

```
SRC = $(wildcard *.c)
OBJ = $(patsubst %.c,%.o,$(SRC))
```

```
wildcard    -> finds matching files
patsubst    -> transforms names
filter      -> selects matching words
filter-out  -> removes matching words
```

6. Assignment Operators

Operator	Meaning	Example
=	Recursive assignment	SRC = \$(wildcard *.c)
:=	Immediate assignment	OBJ := \$(patsubst %.c,%o,\$(SRC))
+=	Append	CFLAGS += -g
?=	Default if not already defined	DEBUG ?= yes

7. Conditionals

```
ifeq ($(DEBUG),yes)
CFLAGS += -g
endif

ifneq ($(DEBUG),yes)
CFLAGS += -O2
endif

ifdef DEBUG
# variable is defined
endif

ifndef DEBUG
# variable is not defined
endif
```

ifeq/ifneq compare values. ifdef/ifndef check whether a variable is defined. For yes/no feature selection, ifeq is usually clearest.

8. Feature Enable / Disable

```
DEBUG      ?= yes
ENABLE_ADD ?= yes
ENABLE_SUB ?= no

ifeq ($(DEBUG),yes)
CFLAGS += -g
endif

SRC = $(wildcard *.c)

ifeq ($(ENABLE_ADD),no)
SRC := $(filter-out add.c,$(SRC))
endif

ifeq ($(ENABLE_SUB),no)
SRC := $(filter-out sub.c,$(SRC))
endif

OBJ := $(patsubst %.c,%o,$(SRC))
```

9. .config Concept

```
.config
CONFIG_DEBUG=y
CONFIG_ADD=y
CONFIG_SUB=n

Makefile:
include .config

ifeq ($(CONFIG_DEBUG),y)
CFLAGS += -g
endif

ifeq ($(CONFIG_ADD),n)
SRC := $(filter-out add.c,$(SRC))
endif

ifeq ($(CONFIG_SUB),n)
SRC := $(filter-out sub.c,$(SRC))
```

endif

Flow: .config stores settings → include .config imports them → conditionals interpret CONFIG_xxx=y/n → flags/source lists change → build changes. This is a simplified learning model, not the full Linux Kconfig/Kbuild implementation.

10. CFLAGS / CPPFLAGS / LDFLAGS / LDLIBS

Variable	Purpose	Examples
CFLAGS	C compiler options	-Wall -O2 -g
CPPFLAGS	Preprocessor options	-linclude -DDEBUG
LDFLAGS	Linker options/search paths	-L./lib
LDLIBS	Libraries to link	-lm -lpthread

Compile:
gcc \$(CPPFLAGS) \$(CFLAGS) -c main.c -o main.o

Link:
gcc \$(LDFLAGS) main.o add.o \$(LDLIBS) -o SampleKlcTest

11. .PHONY

.PHONY: all clean install uninstall print

Use .PHONY for action targets that are not real files, such as clean or print.

12. Useful make Commands

Command	Purpose
make	Build default target
make print	Inspect variables using a custom print target
make clean	Remove generated files according to the rule
make -n	Show commands without executing them
make -B	Force rebuilding
make -j4	Allow up to four parallel jobs
make VAR=value	Override a variable from the command line

13. Complete Practical Example

```
TARGET = SampleKlcTest
CC = gcc

include .config

SRC = $(wildcard *.c)
CFLAGS =

ifeq ($(CONFIG_DEBUG),y)
CFLAGS += -g
endif

ifeq ($(CONFIG_ADD),n)
SRC := $(filter-out add.c,$(SRC))
endif

ifeq ($(CONFIG_SUB),n)
SRC := $(filter-out sub.c,$(SRC))
endif

OBJ := $(patsubst %.c,%.o,$(SRC))

.PHONY: all clean print install uninstall

all: $(TARGET)

$(TARGET): $(OBJ)
<TAB>$(CC) $(LDFLAGS) $^ $(LDLIBS) -o $@

%.o: %.c
<TAB>$(CC) $(CPPFLAGS) $(CFLAGS) -c $< -o $@

print:
<TAB>@echo "SRC = $(SRC)"
<TAB>@echo "OBJ = $(OBJ)"
<TAB>@echo "CFLAGS = $(CFLAGS)"
<TAB>@echo "CONFIG_DEBUG = $(CONFIG_DEBUG)"
<TAB>@echo "CONFIG_ADD = $(CONFIG_ADD)"
<TAB>@echo "CONFIG_SUB = $(CONFIG_SUB)"

clean:
<TAB>rm -f *.o $(TARGET)

install:
<TAB>cp $(TARGET) /bin/

uninstall:
<TAB>rm -f /bin/$(TARGET)
```

14. Build Flow — Quick Reference

```
.config
|
| include
v
Makefile configuration + conditionals
|
+--> wildcard --> source list
|
+--> filter-out --> remove disabled features
|
+--> patsubst --> object list
|
v
%.o: %.c
|
| CPPFLAGS + CFLAGS
v
Object files
|
| LDFLAGS + LDLIBS
v
Executable
```

15. Embedded Linux Focus

For board bring-up and BSP/application work, the useful skill is to read an existing build system, trace configuration → source/object selection → compilation → linking, and safely make small changes. Advanced Make programming is not required for this learning path.

16. Quick Memory Sheet

`$@` = target
`$<` = first prerequisite
`$^` = all prerequisites

`wildcard` = find files
`patsubst` = transform names
`filter` = select
`filter-out` = remove

`ifeq / ifneq` = compare values
`ifdef / ifndef` = test whether defined

`=` = recursive assignment
`:=` = immediate assignment
`+=` = append
`?=` = default if undefined

`.config` + `include` + `CONFIG_xxx=y/n`
↓
configuration-driven build